

OpenROAD is an open-source, fully automated **RTL-to-GDSII** toolchain for **digital IC (Integrated Circuit) design**. It offers an end-to-end physical design flow — from register-transfer level (RTL) synthesis to final layout (GDSII) output — supporting synthesis, floorplanning, placement, clock tree synthesis, routing, and layout generation. Its modular, scriptable, and GUI-supported environment makes it suitable for both **academic research** and **industrial prototyping**.

Installation Overview

The installation and setup of OpenROAD are structured into several well-defined steps:

Step 1 – Install Prerequisites:

Update your system and install required build tools and libraries such as *CMake*, *Clang*, *G++*, *Python3*, *Boost*, *SWIG*, *Tcl*, and others.

Step 2 – Clone Repositories:

Obtain the OpenROAD-flow-scripts Git repository (which wraps tools like Yosys and OpenROAD) by cloning it recursively.

Step 3 – Run Setup Script:

Execute the `setup.sh` script to download, configure, and install all third-party dependencies, including Boost, Abseil, and SWIG.

Step 4 – Build OpenROAD Locally:

Use the automated `build_openroad.sh --local` command to compile OpenROAD. Optionally disable tests to speed up builds.

Step 5 – Verify Installation:

Source the environment file (`source ./env.sh`) and verify the installation of OpenROAD, Yosys, and Verilator using their `--version` and `-help` options.

Step 6 – Run the OpenROAD Flow:

Enter the `flow` directory and execute `make` to run the standard flow from RTL to GDSII.

Step 7 – Launch GUI:

Start the graphical interface with `make gui_final` to visualize physical layouts and analyze design results.

Once installed, OpenROAD gives access to the full flow and GUI-based visualization capabilities for physical design exploration.

Directory Structure and File Organization

The **OpenROAD-flow-scripts** repository is organized for modular functionality:

OpenROAD-flow-scripts/

— docker/	-> Docker setup and run scripts
— docs/	-> Documentation and references
— flow/	-> RTL-to-GDSII flow automation files

- |— jenkins/ -> Regression tests for builds
- |— tools/ -> Core design tools required for RTL-to-GDS flow
- |— etc/ -> Environment and dependency setup scripts
- |— setup_env.sh -> Master environment configuration script

Inside the **flow** directory:

flow/

- |— design/ -> Predefined RTL-to-GDSII design examples
- |— makefile -> Flow automation driver
- |— platform/ -> Technology-specific libraries, LEFs, and GDS files
- |— tutorials/ -> Example tutorials
- |— util/
- |— scripts/

Floorplanning and Placement Features

OpenROAD supports interactive and script-driven **floorplanning**, **pin placement**, and **macro/standard cell placement** operations via GUI and Tcl scripting.

Floorplan Initialization

You can initialize floorplans by specifying:

Core and die area dimensions, or

Core utilization and aspect ratio.

Commands like `initialize_floorplan` define these parameters, while `.tcl` scripts automate loading LEF, liberty, and Verilog files, linking design, and generating DEF outputs.

Example Floorplan Methods

By core and die area: Specifies exact coordinates for die and core boundaries.

By core utilization: Uses utilization, aspect ratio, and core spacing values to compute area distribution automatically.

Both methods visualize results through OpenROAD's GUI with commands:

```
openroad -gui
```

```
source init_floorplan1.tcl
```

IO Pin Placement

Pin placement scripts (place_pin4.tcl) position pins along die boundaries to minimize wirelength. Pins are assigned to metal layers with defined coordinates and sizes, and final layouts are reviewed using:

```
openroad -gui
```

```
source place_pin4.tcl
```

This process ensures optimal routing and signal accessibility.

Macro and Standard Cell Placement

OpenROAD enables **global placement** for macros and standard cells to achieve efficient area usage and timing optimization. Placement density directly affects core spreading and timing results. Scripts like macro01.tcl load LEF/DEF files, run placement, and generate results displayed in GUI.

Command example:

```
openroad -gui
```

```
source macro01.tcl
```

You can then analyze HPWL (Half-Perimeter Wire Length), placement efficiency, and physical distribution of standard cells and macros.

Thank you for VSD Team for this RSIC V Tapeout Program Dr M A Raheem MJCET Hyderabad